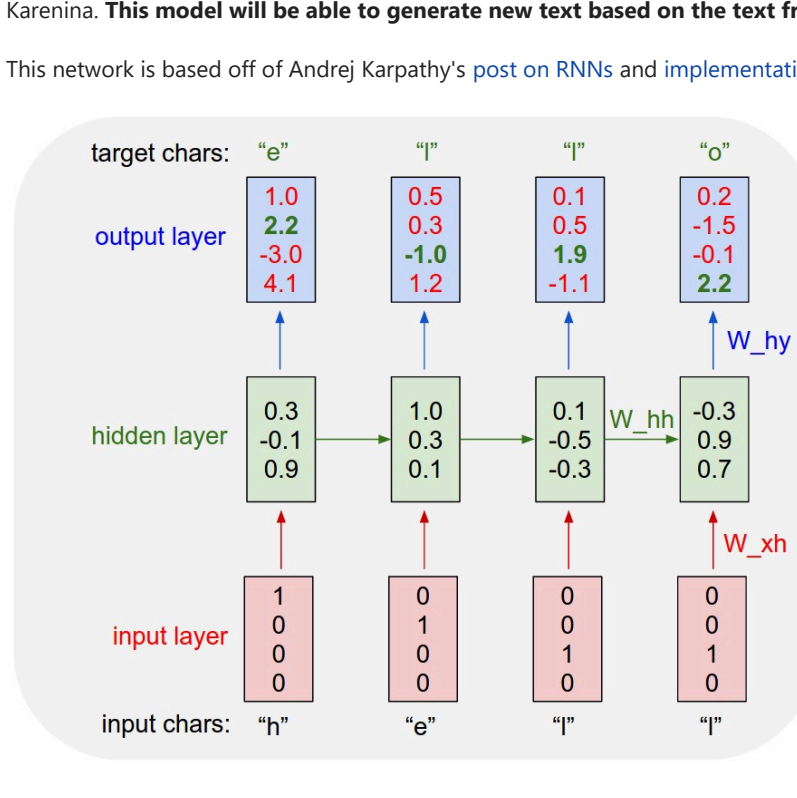
Character-Level LSTM in PyTorch

In this notebook, I'll construct a character-level LSTM with PyTorch. The network will train character by character on some text, then generate new text character by character. As an example, I will train on Anna Karenina. **This model will be able to generate new text based on the text from the book!**

This network is based off of Andrej Karpathy's post on RNNs and implementation in Torch. Below is the general architecture of the character-wise RNN.



First let's load in our required resources for data loading and model creation.

```
In [2]: import numpy as np
        import torch
        from torch import nn
        import torch.nn.functional as F
```

Load in Data

Then, we'll load the Anna Karenina text file and convert it into integers for our network to use.

```
In [3]: # open text file and read in data as `text`
        with open('data/anna.txt', 'r') as f:
            text = f.read()
```

Let's check out the first 100 characters, make sure everything is peachy. According to the American Book Review, this is the 6th best first line of a book ever.

```
In [4]: text[:100]

Out[4]: 'Chapter 1\n\n\nHappy families are all alike; every unhappy family is unhappy in its own\nway.\n\nEverythin'
```

Tokenization

In the cells, below, I'm creating a couple of **dictionaries** to convert the characters to and from integers. Encoding the characters as integers makes it easier to use as input in the network.

```
In [5]: # encode the text and map each character to an integer and vice versa

        # we create two dictionaries:
        # 1. int2char, which maps integers to characters
        # 2. char2int, which maps characters to unique integers
        chars = tuple(set(text))
        int2char = dict(enumerate(chars))
        char2int = {ch: ii for ii, ch in int2char.items()}

        # encode the text
        encoded = np.array([char2int[ch] for ch in text])
```

And we can see those same characters from above, encoded as integers.

```
In [6]: encoded[:100]

Out[6]: array([73, 32, 64, 19, 40, 74, 27, 29, 38, 72, 72, 72, 13, 64, 19, 19, 82,
               29, 26, 64, 11, 53, 78, 53, 74, 60, 29, 64, 27, 74, 29, 64, 78, 78,
               29, 64, 78, 53, 28, 74, 77, 29, 74, 18, 74, 27, 82, 29, 58, 15, 32,
               64, 19, 19, 82, 29, 26, 64, 11, 53, 78, 82, 29, 53, 60, 29, 58, 15,
               32, 64, 19, 19, 82, 29, 53, 15, 29, 53, 40, 60, 29, 36, 16, 15, 72,
               16, 64, 82,  3, 72, 72, 76, 18, 74, 27, 82, 40, 32, 53, 15])
```

Pre-processing the data

As you can see in our char-RNN image above, our LSTM expects an input that is **one-hot encoded** meaning that each character is converted into an integer (via our created dictionary) and *then* converted into a column vector where only it's corresponding integer index will have the value of 1 and the rest of the vector will be filled with 0's. Since we're one-hot encoding the data, let's make a function to do that!

```
In [7]: def one_hot_encode(arr, n_labels):

            # Initialize the encoded array
            one_hot = np.zeros((arr.size, n_labels), dtype=np.float32)

            # Fill the appropriate elements with ones
            one_hot[np.arange(one_hot.shape[0]), arr.flatten()] = 1.

            # Finally reshape it to get back to the original array
            one_hot = one_hot.reshape((*arr.shape, n_labels))

            return one_hot
```

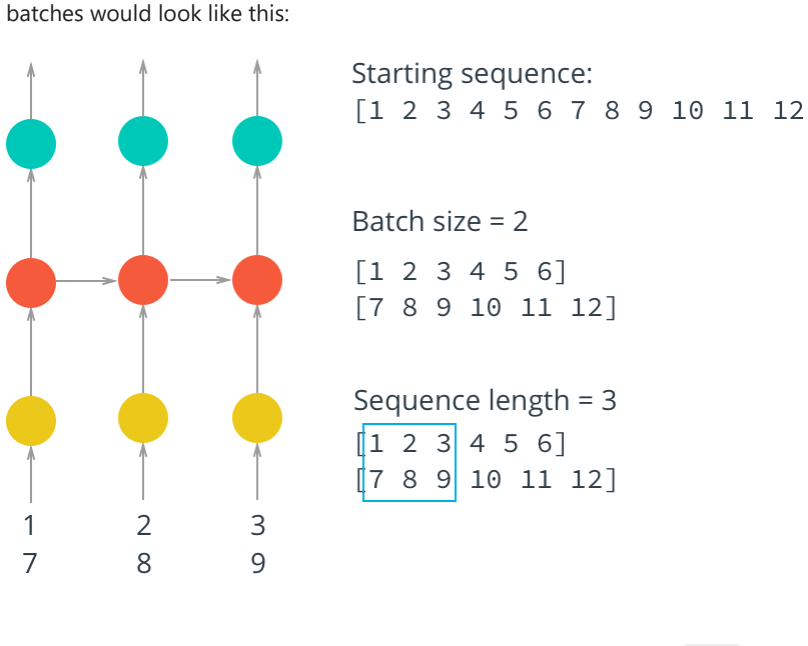
```
In [8]: # check that the function works as expected
        test_seq = np.array([[3, 5, 1]])
        one_hot = one_hot_encode(test_seq, 8)

        print(one_hot)

[[[0. 0. 0. 1. 0. 0. 0. 0.]
  [0. 0. 0. 0. 0. 1. 0. 0.]
  [0. 1. 0. 0. 0. 0. 0. 0.]]]
```

Making training mini-batches

To train on this data, we also want to create mini-batches for training. Remember that we want our batches to be multiple sequences of some desired number of sequence steps. Considering a simple example, our batches would look like this:



In this example, we'll take the encoded characters (passed in as the `arr` parameter) and split them into multiple sequences, given by `batch_size`. Each of our sequences will be `seq_length` long.

Creating Batches

**1. The first thing we need to do is discard some of the text so we only have completely full mini-batches. **

Each batch contains $N \times M$ characters, where N is the batch size (the number of sequences in a batch) and M is the seq_length or number of time steps in a sequence. Then, to get the total number of batches, K, that we can make from the array `arr`, you divide the length of `arr` by the number of characters per batch. Once you know the number of batches, you can get the total number of characters to keep from `arr`, $N * M * K$.

**2. After that, we need to split `arr` into N batches. **

You can do this using `arr.reshape(size)` where `size` is a tuple containing the dimensions sizes of the reshaped array. We know we want N sequences in a batch, so let's make that the size of the first dimension. For the second dimension, you can use `-1` as a placeholder in the size, it'll fill up the array with the appropriate data for you. After this, you should have an array that is $N \times (M * K)$.

**3. Now that we have this array, we can iterate through it to get our mini-batches. **

The idea is each batch is a $N \times M$ window on the $N \times (M * K)$ array. For each subsequent batch, the window moves over by `seq_length`. We also want to create both the input and target arrays. Remember that the targets are just the inputs shifted over by one character. The way I like to do this window is use `range` to take steps of size `n_steps` from 0 to `arr.shape[1]`, the total number of tokens in each sequence. That way, the integers you get from `range` always point to the start of a batch, and each window is `seq_length` wide.

> **TODO:** Write the code for creating batches in the function below. The exercises in this notebook *will not be easy*. I've provided a notebook with solutions alongside this notebook. If you get stuck, checkout the solutions. The most important thing is that you don't copy and paste the code into here, **type out the solution code yourself.**

```
In [9]: def get_batches(arr, batch_size, seq_length):
            '''Create a generator that returns batches of size
               batch_size x seq_length from arr.

               Arguments
               ---------
               arr: Array you want to make batches from
               batch_size: Batch size, the number of sequences per batch
               seq_length: Number of encoded chars in a sequence
            '''

            batch_size_total = batch_size * seq_length
            # total number of batches we can make
            n_batches = len(arr)//batch_size_total

            # Keep only enough characters to make full batches
            arr = arr[:n_batches * batch_size_total]
            # Reshape into batch_size rows
            arr = arr.reshape((batch_size, -1))

            # iterate through the array, one sequence at a time
            for n in range(0, arr.shape[1], seq_length):
                # The features
                x = arr[:, n:n+seq_length]
                # The targets, shifted by one
                y = np.zeros_like(x)
                try:
                    y[:, :-1], y[:, -1] = x[:, 1:], arr[:, n+seq_length]
                except IndexError:
                    y[:, :-1], y[:, -1] = x[:, 1:], arr[:, 0]
                yield x, y
```

Test Your Implementation

Now I'll make some data sets and we can check out what's going on as we batch data. Here, as an example, I'm going to use a batch size of 8 and 50 sequence steps.

```
In [10]: batches = get_batches(encoded, 8, 50)
         x, y = next(batches)
```

```
In [11]: # printing out the first 10 items in a sequence
         print('x\n', x[:10, :10])
         print('\ny\n', y[:10, :10])

x
 [[73 32 64 19 40 74 27 29 38 72]
 [60 36 15 29 40 32 64 40 29 64]
 [74 15 61 29 36 27 29 64 29 26]
 [60 29 40 32 74 29 14 32 53 74]
 [29 60 64 16 29 32 74 27 29 40]
 [14 58 60 60 53 36 15 29 64 15]
 [29 75 15 15 64 29 32 64 61 29]
 [47 59 78 36 15 60 28 82  3 29]]

y
 [[32 64 19 40 74 27 29 38 72 72]
 [36 15 29 40 32 64 40 29 64 40]
 [15 61 29 36 27 29 64 29 26 36]
 [29 40 32 74 29 14 32 53 74 26]
 [60 64 16 29 32 74 27 29 40 74]
 [58 60 60 53 36 15 29 64 15 61]
 [75 15 15 64 29 32 64 61 29 60]
 [59 78 36 15 60 28 82  3 29 24]]
```

If you implemented `get_batches` correctly, the above output should look something like

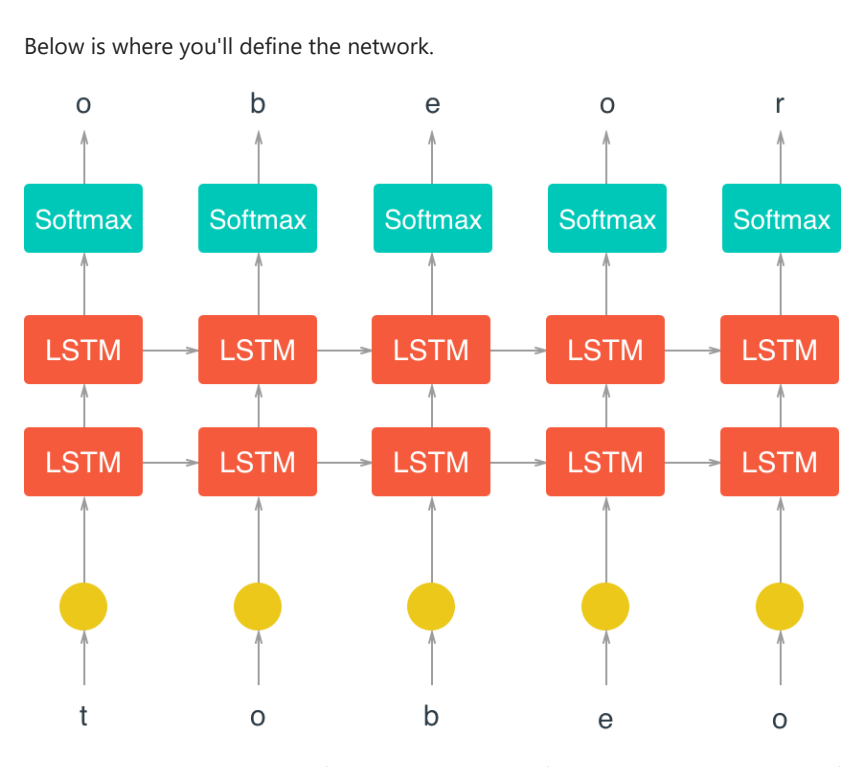
```
x
 [[25  8 60 11 45 27 28 73  1  2]
 [17  7 20 73 45  8 60 45 73 60]
 [27 20 80 73  7 28 73 60 73 65]
 [17 73 45  8 27 73 66  8 46 27]
 [73 17 60 12 73  8 27 28 73 45]
 [66 64 17 17 46  7 20 73 60 20]
 [73 76 20 20 60 73  8 60 80 73]
 [47 35 43  7 20 17 24 50 37 73]]

y
 [[ 8 60 11 45 27 28 73  1  2  2]
 [ 7 20 73 45  8 60 45 73 60 45]
 [20 80 73  7 28 73 60 73 65  7]
 [73 45  8 27 73 66  8 46 27 65]
 [17 60 12 73  8 27 28 73 45 27]
 [64 17 17 46  7 20 73 60 20 80]
 [76 20 20 60 73  8 60 80 73 17]
 [35 43  7 20 17 24 50 37 73 36]]
```

although the exact numbers may be different. Check to make sure the data is shifted over one step for `y`.

Defining the network with PyTorch

Below is where you'll define the network.



Next, you'll use PyTorch to define the architecture of the network. We start by defining the layers and operations we want. Then, define a method for the forward pass. You've also been given a method for predicting characters.

Model Structure

In `__init__` the suggested structure is as follows:

- Create and store the necessary dictionaries (this has been done for you)
- Define an LSTM layer that takes as params: an input size (the number of characters), a hidden layer size `n_hidden`, a number of layers `n_layers`, a dropout probability `drop_prob`, and a batch_first boolean (True, since we are batching)
- Define a dropout layer with `drop_prob`
- Define a fully-connected layer with params: input size `n_hidden` and output size (the number of characters)
- Finally, initialize the weights (again, this has been given)

Note that some parameters have been named and given in the `__init__` function, and we use them and store them by doing something like `self.drop_prob = drop_prob`.

LSTM Inputs/Outputs

You can create a basic LSTM layer as follows

```
self.lstm = nn.LSTM(input_size, n_hidden, n_layers,
                            dropout=drop_prob, batch_first=True)
```

where `input_size` is the number of characters this cell expects to see as sequential input, and `n_hidden` is the number of units in the hidden layers in the cell. And we can add dropout by adding a dropout parameter with a specified probability; this will automatically add dropout to the inputs or outputs. Finally, in the `forward` function, we can stack up the LSTM cells into layers using `.view`. With this, you pass in a list of cells and it will send the output of one cell into the next cell.

We also need to create an initial hidden state of all zeros. This is done like so

```
self.init_hidden()
```

```
In [12]: # check if GPU is available
         train_on_gpu = torch.cuda.is_available()
         if(train_on_gpu):
             print('Training on GPU!')
         else:
             print('No GPU available, training on CPU; consider making n_epochs very small.')

Training on GPU!
```

```
In [13]: class CharRNN(nn.Module):

             def __init__(self, tokens, n_hidden=256, n_layers=2,
                                        drop_prob=0.5, lr=0.001):
                 super().__init__()
                 self.drop_prob = drop_prob
                 self.n_layers = n_layers
                 self.n_hidden = n_hidden
                 self.lr = lr

                 # creating character dictionaries
                 self.chars = tokens
                 self.int2char = dict(enumerate(self.chars))
                 self.char2int = {ch: ii for ii, ch in self.int2char.items()}

                 ## TODO: define the LSTM
                 self.lstm = nn.LSTM(len(self.chars), n_hidden, n_layers,
                                     dropout=drop_prob, batch_first=True)

                 ## TODO: define a dropout layer
                 self.dropout = nn.Dropout(drop_prob)

                 ## TODO: define the final, fully-connected output layer
                 self.fc = nn.Linear(n_hidden, len(self.chars))


             def forward(self, x, hidden):
                 ''' Forward pass through the network.
                     These inputs are x, and the hidden/cell state `hidden`. '''

                 ## TODO: Get the outputs and the new hidden state from the lstm
                 r_output, hidden = self.lstm(x, hidden)

                 ## TODO: pass through a dropout layer
                 out = self.dropout(r_output)

                 # Stack up LSTM outputs using view
                 # you may need to use contiguous to reshape the output
                 out = out.contiguous().view(-1, self.n_hidden)

                 ## TODO: put x through the fully-connected layer
                 out = self.fc(out)

                 # return the final output and the hidden state
                 return out, hidden


             def init_hidden(self, batch_size):
                 ''' Initializes hidden state '''
                 # Create two new tensors with sizes n_layers x batch_size x n_hidden,
                 # initialized to zero, for hidden state and cell state of LSTM
                 weight = next(self.parameters()).data

                 if (train_on_gpu):
                     hidden = (weight.new(self.n_layers, batch_size, self.n_hidden).zero_().cuda(),
                               weight.new(self.n_layers, batch_size, self.n_hidden).zero_().cuda())
                 else:
                     hidden = (weight.new(self.n_layers, batch_size, self.n_hidden).zero_(),
                               weight.new(self.n_layers, batch_size, self.n_hidden).zero_())

                 return hidden
```

Time to train

The train function gives us the ability to set the number of epochs, the learning rate, and other parameters.

Below we're using an Adam optimizer and cross entropy loss since we are looking at character class scores as output. We calculate the loss and perform backpropagation, as usual!

A couple of details about training:

> - Within the batch loop, we detach the hidden state from its history; this time setting it equal to a new tuple variable because an LSTM has a hidden state that is a tuple of the hidden and cell states.

- We use `clip_grad_norm_` to help prevent exploding gradients.

```
In [14]: def train(net, data, epochs=10, batch_size=10, seq_length=50, lr=0.001, clip=5, val_frac=0.1, print_every=10):
             ''' Training a network

                 Arguments
                 ---------

                 net: CharRNN network
                 data: text data to train the network
                 epochs: Number of epochs to train
                 batch_size: Number of mini-sequences per mini-batch, aka batch size
                 seq_length: Number of character steps per mini-batch
                 lr: learning rate
                 clip: gradient clipping
                 val_frac: Fraction of data to hold out for validation
                 print_every: Number of steps for printing training and validation loss

             '''
             net.train()

             opt = torch.optim.Adam(net.parameters(), lr=lr)
             criterion = nn.CrossEntropyLoss()

             # create training and validation data
             val_idx = int(len(data)*(1-val_frac))
             data, val_data = data[:val_idx], data[val_idx:]

             if(train_on_gpu):
                 net.cuda()

             counter = 0
             n_chars = len(net.chars)
             for e in range(epochs):
                 # initialize hidden state
                 h = net.init_hidden(batch_size)

                 for x, y in get_batches(data, batch_size, seq_length):
                     counter += 1

                     # One-hot encode our data and make them Torch tensors
                     x = one_hot_encode(x, n_chars)
                     inputs, targets = torch.from_numpy(x), torch.from_numpy(y)

                     if(train_on_gpu):
                         inputs, targets = inputs.cuda(), targets.cuda()

                     # Creating new variables for the hidden state, otherwise
                     # we'd backprop through the entire training history
                     h = tuple([each.data for each in h])

                     # zero accumulated gradients
                     net.zero_grad()

                     # get the output from the model
                     output, h = net(inputs, h)

                     # calculate the loss and perform backprop
                     loss = criterion(output, targets.view(batch_size*seq_length).long())
                     loss.backward()
                     # `clip_grad_norm` helps prevent the exploding gradient problem in RNNs / LSTMs.
                     nn.utils.clip_grad_norm_(net.parameters(), clip)
                     opt.step()

                     # loss stats
                     if counter % print_every == 0:
                         # Get validation loss
                         val_h = net.init_hidden(batch_size)
                         val_losses = []
                         net.eval()
                         for x, y in get_batches(val_data, batch_size, seq_length):
                             # One-hot encode our data and make them Torch tensors
                             x = one_hot_encode(x, n_chars)
                             x, y = torch.from_numpy(x), torch.from_numpy(y)

                             # Creating new variables for the hidden state, otherwise
                             # we'd backprop through the entire training history
                             val_h = tuple([each.data for each in val_h])

                             inputs, targets = x, y
                             if(train_on_gpu):
                                 inputs, targets = inputs.cuda(), targets.cuda()

                             output, val_h = net(inputs, val_h)
                             val_loss = criterion(output, targets.view(batch_size*seq_length).long())

                             val_losses.append(val_loss.item())

                         net.train() # reset to train mode after iterationg through validation data

                         print("Epoch: {}/{}...".format(e+1, epochs),
                               "Step: {}...".format(counter),
                               "Loss: {:.4f}...".format(loss.item()),
                               "Val Loss: {:.4f}".format(np.mean(val_losses)))
```

Instantiating the model

Now we can actually train the network. First we'll create the network itself, with some given hyperparameters. Then, define the mini-batches sizes, and start training!

```
In [15]: # define and print the net
         n_hidden=512
         n_layers=2

         net = CharRNN(chars, n_hidden, n_layers)
         print(net)

CharRNN(
  (lstm): LSTM(83, 512, num_layers=2, batch_first=True, dropout=0.5)
  (dropout): Dropout(p=0.5, inplace=False)
  (fc): Linear(in_features=512, out_features=83, bias=True)
)
```

Set your training hyperparameters!

```
In [16]: batch_size = 128
         seq_length = 100
         n_epochs = 20 # start smaller if you are just testing initial behavior

         # train the model
         train(net, encoded, epochs=n_epochs, batch_size=batch_size, seq_length=seq_length, lr=0.001, print_every=10)

Epoch: 1/20... Step: 10... Loss: 3.2688... Val Loss: 3.2043
Epoch: 1/20... Step: 20... Loss: 3.1583... Val Loss: 3.1342
Epoch: 1/20... Step: 30... Loss: 3.1445... Val Loss: 3.1222
Epoch: 1/20... Step: 40... Loss: 3.1138... Val Loss: 3.1188
Epoch: 1/20... Step: 50... Loss: 3.1439... Val Loss: 3.1173
Epoch: 1/20... Step: 60... Loss: 3.1218... Val Loss: 3.1157
Epoch: 1/20... Step: 70... Loss: 3.1111... Val Loss: 3.1134
Epoch: 1/20... Step: 80... Loss: 3.1207... Val Loss: 3.1083
Epoch: 1/20... Step: 90... Loss: 3.1098... Val Loss: 3.0954
Epoch: 1/20... Step: 100... Loss: 3.0786... Val Loss: 3.0649
Epoch: 1/20... Step: 110... Loss: 3.0431... Val Loss: 3.0245
Epoch: 1/20... Step: 120... Loss: 2.9701... Val Loss: 2.9766
Epoch: 1/20... Step: 130... Loss: 2.8853... Val Loss: 2.8523
Epoch: 2/20... Step: 140... Loss: 2.7824... Val Loss: 2.7448
Epoch: 2/20... Step: 150... Loss: 2.6943... Val Loss: 2.6296
Epoch: 2/20... Step: 160... Loss: 2.5881... Val Loss: 2.5553
Epoch: 2/20... Step: 170... Loss: 2.5209... Val Loss: 2.4991
Epoch: 2/20... Step: 180... Loss: 2.4814... Val Loss: 2.4592
Epoch: 2/20... Step: 190... Loss: 2.4323... Val Loss: 2.4176
Epoch: 2/20... Step: 200... Loss: 2.4227... Val Loss: 2.3858
Epoch: 2/20... Step: 210... Loss: 2.3849... Val Loss: 2.3615
Epoch: 2/20... Step: 220... Loss: 2.3476... Val Loss: 2.3289
Epoch: 2/20... Step: 230... Loss: 2.3417... Val Loss: 2.3019
Epoch: 2/20... Step: 240... Loss: 2.3231... Val Loss: 2.2832
Epoch: 2/20... Step: 250... Loss: 2.2594... Val Loss: 2.2556
Epoch: 2/20... Step: 260... Loss: 2.2342... Val Loss: 2.2299
Epoch: 2/20... Step: 270... Loss: 2.2338... Val Loss: 2.2059
Epoch: 3/20... Step: 280... Loss: 2.2334... Val Loss: 2.1834
Epoch: 3/20... Step: 290... Loss: 2.1954... Val Loss: 2.1589
Epoch: 3/20... Step: 300... Loss: 2.1654... Val Loss: 2.1380
Epoch: 3/20... Step: 310... Loss: 2.1415... Val Loss: 2.1227
Epoch: 3/20... Step: 320... Loss: 2.1101... Val Loss: 2.0994
Epoch: 3/20... Step: 330... Loss: 2.0935... Val Loss: 2.0823
Epoch: 3/20... Step: 340... Loss: 2.1030... Val Loss: 2.0611
Epoch: 3/20... Step: 350... Loss: 2.0775... Val Loss: 2.0417
Epoch: 3/20... Step: 360... Loss: 2.0157... Val Loss: 2.0221
Epoch: 3/20... Step: 370... Loss: 2.0425... Val Loss: 2.0081
Epoch: 3/20... Step: 380... Loss: 2.0077... Val Loss: 1.9891
Epoch: 3/20... Step: 390... Loss: 1.9837... Val Loss: 1.9762
Epoch: 3/20... Step: 400... Loss: 1.9613... Val Loss: 1.9592
Epoch: 3/20... Step: 410... Loss: 1.9766... Val Loss: 1.9485
Epoch: 4/20... Step: 420... Loss: 1.9705... Val Loss: 1.9310
Epoch: 4/20... Step: 430... Loss: 1.9517... Val Loss: 1.9151
Epoch: 4/20... Step: 440... Loss: 1.9325... Val Loss: 1.9039
Epoch: 4/20... Step: 450... Loss: 1.8745... Val Loss: 1.8918
Epoch: 4/20... Step: 460... Loss: 1.8752... Val Loss: 1.8804
Epoch: 4/20... Step: 470... Loss: 1.8932... Val Loss: 1.8732
Epoch: 4/20... Step: 480... Loss: 1.8828... Val Loss: 1.8568
Epoch: 4/20... Step: 490... Loss: 1.8847... Val Loss: 1.8473
Epoch: 4/20... Step: 500... Loss: 1.8769... Val Loss: 1.8330
Epoch: 4/20... Step: 510... Loss: 1.8585... Val Loss: 1.8240
Epoch: 4/20... Step: 520... Loss: 1.8566... Val Loss: 1.8089
Epoch: 4/20... Step: 530... Loss: 1.8296... Val Loss: 1.8019
Epoch: 4/20... Step: 540... Loss: 1.7952... Val Loss: 1.7922
Epoch: 4/20... Step: 550... Loss: 1.8354... Val Loss: 1.7804
Epoch: 5/20... Step: 560... Loss: 1.7971... Val Loss: 1.7718
Epoch: 5/20... Step: 570... Loss: 1.7836... Val Loss: 1.7615
Epoch: 5/20... Step: 580... Loss: 1.7616... Val Loss: 1.7552
Epoch: 5/20... Step: 590... Loss: 1.7674... Val Loss: 1.7450
Epoch: 5/20... Step: 600... Loss: 1.7528... Val Loss: 1.7389
Epoch: 5/20... Step: 610... Loss: 1.7297... Val Loss: 1.7323
Epoch: 5/20... Step: 620... Loss: 1.7496... Val Loss: 1.7304
Epoch: 5/20... Step: 630... Loss: 1.7597... Val Loss: 1.7164
Epoch: 5/20... Step: 640... Loss: 1.7244... Val Loss: 1.7073
Epoch: 5/20... Step: 650... Loss: 1.7193... Val Loss: 1.6997
Epoch: 5/20... Step: 660... Loss: 1.6882... Val Loss: 1.6919
Epoch: 5/20... Step: 670... Loss: 1.7189... Val Loss: 1.6856
Epoch: 5/20... Step: 680... Loss: 1.7072... Val Loss: 1.6796
Epoch: 5/20... Step: 690... Loss: 1.6855... Val Loss: 1.6745
Epoch: 6/20... Step: 700... Loss: 1.7002... Val Loss: 1.6738
Epoch: 6/20... Step: 710... Loss: 1.6791... Val Loss: 1.6605
Epoch: 6/20... Step: 720... Loss: 1.6701... Val Loss: 1.6520
Epoch: 6/20... Step: 730... Loss: 1.6812... Val Loss: 1.6473
Epoch: 6/20... Step: 740... Loss: 1.6405... Val Loss: 1.6427
Epoch: 6/20... Step: 750... Loss: 1.6288... Val Loss: 1.6377
Epoch: 6/20... Step: 760... Loss: 1.6638... Val Loss: 1.6347
Epoch: 6/20... Step: 770... Loss: 1.6443... Val Loss: 1.6287
Epoch: 6/20... Step: 780... Loss: 1.6286... Val Loss: 1.6210
Epoch: 6/20... Step: 790... Loss: 1.6190... Val Loss: 1.6169
Epoch: 6/20... Step: 800... Loss: 1.6401... Val Loss: 1.6127
Epoch: 6/20... Step: 810... Loss: 1.6209... Val Loss: 1.6128
Epoch: 6/20... Step: 820... Loss: 1.5868... Val Loss: 1.6080
Epoch: 6/20... Step: 830... Loss: 1.6328... Val Loss: 1.6003
Epoch: 7/20... Step: 840... Loss: 1.5808... Val Loss: 1.5939
Epoch: 7/20... Step: 850... Loss: 1.5967... Val Loss: 1.5886
Epoch: 7/20... Step: 860... Loss: 1.5882... Val Loss: 1.5820
Epoch: 7/20... Step: 870... Loss: 1.5847... Val Loss: 1.5783
Epoch: 7/20... Step: 880... Loss: 1.5968... Val Loss: 1.5760
Epoch: 7/20... Step: 890... Loss: 1.5959... Val Loss: 1.5716
Epoch: 7/20... Step: 900... Loss: 1.5675... Val Loss: 1.5744
Epoch: 7/20... Step: 910... Loss: 1.5482... Val Loss: 1.5661
Epoch: 7/20... Step: 920... Loss: 1.5730... Val Loss: 1.5613
Epoch: 7/20... Step: 930... Loss: 1.5489... Val Loss: 1.5551
Epoch: 7/20... Step: 940... Loss: 1.5616... Val Loss: 1.5556
Epoch: 7/20... Step: 950... Loss: 1.5629... Val Loss: 1.5519
Epoch: 7/20... Step: 960... Loss: 1.5598... Val Loss: 1.5470
Epoch: 7/20... Step: 970... Loss: 1.5702... Val Loss: 1.5440
Epoch: 8/20... Step: 980... Loss: 1.5428... Val Loss: 1.5384
Epoch: 8/20... Step: 990... Loss: 1.5485... Val Loss: 1.5360
Epoch: 8/20... Step: 1000... Loss: 1.5249... Val Loss: 1.5282
Epoch: 8/20... Step: 1010... Loss: 1.5730... Val Loss: 1.5276
Epoch: 8/20... Step: 1020... Loss: 1.5433... Val Loss: 1.5226
Epoch: 8/20... Step: 1030... Loss: 1.5239... Val Loss: 1.5205
Epoch: 8/20... Step: 1040... Loss: 1.5363... Val Loss: 1.5196
Epoch: 8/20... Step: 1050... Loss: 1.5129... Val Loss: 1.5154
Epoch: 8/20... Step: 1060... Loss: 1.5218... Val Loss: 1.5147
Epoch: 8/20... Step: 1070... Loss: 1.5175... Val Loss: 1.5094
Epoch: 8/20... Step: 1080... Loss: 1.5172... Val Loss: 1.5074
Epoch: 8/20... Step: 1090... Loss: 1.4950... Val Loss: 1.5044
Epoch: 8/20... Step: 1100... Loss: 1.4912... Val Loss: 1.4972
Epoch: 8/20... Step: 1110... Loss: 1.4922... Val Loss: 1.4977
Epoch: 9/20... Step: 1120... Loss: 1.5064... Val Loss: 1.4950
Epoch: 9/20... Step: 1130... Loss: 1.4986... Val Loss: 1.4906
Epoch: 9/20... Step: 1140... Loss: 1.5048... Val Loss: 1.4845
Epoch: 9/20... Step: 1150... Loss: 1.5176... Val Loss: 1.4845
Epoch: 9/20... Step: 1160... Loss: 1.4805... Val Loss: 1.4830
Epoch: 9/20... Step: 1170... Loss: 1.4748... Val Loss: 1.4831
Epoch: 9/20... Step: 1180... Loss: 1.4777... Val Loss: 1.4824
Epoch: 9/20... Step: 1190... Loss: 1.5084... Val Loss: 1.4760
Epoch: 9/20... Step: 1200... Loss: 1.4516... Val Loss: 1.4792
Epoch: 9/20... Step: 1210... Loss: 1.4779... Val Loss: 1.4728
Epoch: 9/20... Step: 1220... Loss: 1.4644... Val Loss: 1.4692
Epoch: 9/20... Step: 1230... Loss: 1.4511... Val Loss: 1.4712
Epoch: 9/20... Step: 1240... Loss: 1.4556... Val Loss: 1.4619
Epoch: 9/20... Step: 1250... Loss: 1.4668... Val Loss: 1.4616
Epoch: 10/20... Step: 1260... Loss: 1.4691... Val Loss: 1.4629
Epoch: 10/20... Step: 1270... Loss: 1.4577... Val Loss: 1.4558
Epoch: 10/20... Step: 1280... Loss: 1.4738... Val Loss: 1.4514
Epoch: 10/20... Step: 1290... Loss: 1.4607... Val Loss: 1.4540
Epoch: 10/20... Step: 1300... Loss: 1.4505... Val Loss: 1.4502
Epoch: 10/20... Step: 1310... Loss: 1.4618... Val Loss: 1.4486
Epoch: 10/20... Step: 1320... Loss: 1.4154... Val Loss: 1.4496
Epoch: 10/20... Step: 1330... Loss: 1.4246... Val Loss: 1.4451
Epoch: 10/20... Step: 1340... Loss: 1.4026... Val Loss: 1.4430
Epoch: 10/20... Step: 1350... Loss: 1.4191... Val Loss: 1.4414
Epoch: 10/20... Step: 1360... Loss: 1.4171... Val Loss: 1.4397
Epoch: 10/20... Step: 1370... Loss: 1.4060... Val Loss: 1.4394
Epoch: 10/20... Step: 1380... Loss: 1.4461... Val Loss: 1.4333
Epoch: 10/20... Step: 1390... Loss: 1.4531... Val Loss: 1.4301
Epoch: 11/20... Step: 1400... Loss: 1.4450... Val Loss: 1.4305
Epoch: 11/20... Step: 1410... Loss: 1.4706... Val Loss: 1.4290
Epoch: 11/20... Step: 1420... Loss: 1.4389... Val Loss: 1.4232
Epoch: 11/20... Step: 1430... Loss: 1.4220... Val Loss: 1.4256
Epoch: 11/20... Step: 1440... Loss: 1.4391... Val Loss: 1.4230
Epoch: 11/20... Step: 1450... Loss: 1.3721... Val Loss: 1.4212
Epoch: 11/20... Step: 1460... Loss: 1.3984... Val Loss: 1.4226
Epoch: 11/20... Step: 1470... Loss: 1.3875... Val Loss: 1.4200
Epoch: 11/20... Step: 1480... Loss: 1.4096... Val Loss: 1.4164
Epoch: 11/20... Step: 1490... Loss: 1.4059... Val Loss: 1.4126
Epoch: 11/20... Step: 1500... Loss: 1.3920... Val Loss: 1.4166
Epoch: 11/20... Step: 1510... Loss: 1.3805... Val Loss: 1.4124
Epoch: 11/20... Step: 1520... Loss: 1.4081... Val Loss: 1.4064
Epoch: 12/20... Step: 1530... Loss: 1.4565... Val Loss: 1.4095
Epoch: 12/20... Step: 1540... Loss: 1.4149... Val Loss: 1.4054
Epoch: 12/20... Step: 1550... Loss: 1.4219... Val Loss: 1.4042
Epoch: 12/20... Step: 1560... Loss: 1.4303... Val Loss: 1.4015
Epoch: 12/20... Step: 1570... Loss: 1.3772... Val Loss: 1.4039
Epoch: 12/20... Step: 1580... Loss: 1.3492... Val Loss: 1.4009
Epoch: 12/20... Step: 1590... Loss: 1.3466... Val Loss: 1.4006
Epoch: 12/20... Step: 1600... Loss: 1.3723... Val Loss: 1.3986
Epoch: 12/20... Step: 1610... Loss: 1.3518... Val Loss: 1.3991
Epoch: 12/20... Step: 1620... Loss: 1.3607... Val Loss: 1.3972
Epoch: 12/20... Step: 1630... Loss: 1.3881... Val Loss: 1.3950
Epoch: 12/20... Step: 1640... Loss: 1.3691... Val Loss: 1.3956
Epoch: 12/20... Step: 1650... Loss: 1.3373... Val Loss: 1.3917
Epoch: 12/20... Step: 1660... Loss: 1.4019... Val Loss: 1.3882
Epoch: 13/20... Step: 1670... Loss: 1.3603... Val Loss: 1.3876
Epoch: 13/20... Step: 1680... Loss: 1.3781... Val Loss: 1.3846
Epoch: 13/20... Step: 1690... Loss: 1.3495... Val Loss: 1.3830
Epoch: 13/20... Step: 1700... Loss: 1.3514... Val Loss: 1.3827
Epoch: 13/20... Step: 1710... Loss: 1.3388... Val Loss: 1.3849
Epoch: 13/20... Step: 1720... Loss: 1.3451... Val Loss: 1.3826
Epoch: 13/20... Step: 1730... Loss: 1.3851... Val Loss: 1.3807
Epoch: 13/20... Step: 1740... Loss: 1.3455... Val Loss: 1.3814
Epoch: 13/20... Step: 1750... Loss: 1.3203... Val Loss: 1.3814
Epoch: 13/20... Step: 1760... Loss: 1.3455... Val Loss: 1.3817
Epoch: 13/20... Step: 1770... Loss: 1.3634... Val Loss: 1.3771
Epoch: 13/20... Step: 1780... Loss: 1.3383... Val Loss: 1.3757
Epoch: 13/20... Step: 1790... Loss: 1.3348... Val Loss: 1.3751
Epoch: 13/20... Step: 1800... Loss: 1.3557... Val Loss: 1.3747
Epoch: 14/20... Step: 1810... Loss: 1.3508... Val Loss: 1.3767
Epoch: 14/20... Step: 1820... Loss: 1.3376... Val Loss: 1.3716
Epoch: 14/20... Step: 1830... Loss: 1.3579... Val Loss: 1.3694
Epoch: 14/20... Step: 1840... Loss: 1.3087... Val Loss: 1.3678
Epoch: 14/20... Step: 1850... Loss: 1.2973... Val Loss: 1.3676
Epoch: 14/20... Step: 1860... Loss: 1.3480... Val Loss: 1.3710
Epoch: 14/20... Step: 1870... Loss: 1.3538... Val Loss: 1.3655
Epoch: 14/20... Step: 1880... Loss: 1.3391... Val Loss: 1.3683
Epoch: 14/20... Step: 1890... Loss: 1.3549... Val Loss: 1.3681
Epoch: 14/20... Step: 1900... Loss: 1.3327... Val Loss: 1.3623
Epoch: 14/20... Step: 1910... Loss: 1.3399... Val Loss: 1.3619
Epoch: 14/20... Step: 1920... Loss: 1.3290... Val Loss: 1.3649
Epoch: 14/20... Step: 1930... Loss: 1.2884... Val Loss: 1.3654
Epoch: 14/20... Step: 1940... Loss: 1.3571... Val Loss: 1.3578
Epoch: 15/20... Step: 1950... Loss: 1.3320... Val Loss: 1.3630
Epoch: 15/20... Step: 1960... Loss: 1.3270... Val Loss: 1.3546
Epoch: 15/20... Step: 1970... Loss: 1.3173... Val Loss: 1.3559
Epoch: 15/20... Step: 1980... Loss: 1.3169... Val Loss: 1.3573
Epoch: 15/20... Step: 1990... Loss: 1.3076... Val Loss: 1.3543
Epoch: 15/20... Step: 2000... Loss: 1.2891... Val Loss: 1.3553
Epoch: 15/20... Step: 2010... Loss: 1.3144... Val Loss: 1.3512
Epoch: 15/20... Step: 2020... Loss: 1.3404... Val Loss: 1.3522
Epoch: 15/20... Step: 2030... Loss: 1.3037... Val Loss: 1.3534
Epoch: 15/20... Step: 2040... Loss: 1.3183... Val Loss: 1.3482
Epoch: 15/20... Step: 2050... Loss: 1.3019... Val Loss: 1.3502
Epoch: 15/20... Step: 2060... Loss: 1.3156... Val Loss: 1.3475
Epoch: 15/20... Step: 2070... Loss: 1.3200... Val Loss: 1.3476
Epoch: 15/20... Step: 2080... Loss: 1.3106... Val Loss: 1.3452
Epoch: 16/20... Step: 2090... Loss: 1.3109... Val Loss: 1.3457
Epoch: 16/20... Step: 2100... Loss: 1.2998... Val Loss: 1.3438
Epoch: 16/20... Step: 2110... Loss: 1.2933... Val Loss: 1.3441
Epoch: 16/20... Step: 2120... Loss: 1.3077... Val Loss: 1.3440
Epoch: 16/20... Step: 2130... Loss: 1.2823... Val Loss: 1.3433
Epoch: 16/20... Step: 2140... Loss: 1.2849... Val Loss: 1.3407
Epoch: 16/20... Step: 2150... Loss: 1.3215... Val Loss: 1.3387
Epoch: 16/20... Step: 2160... Loss: 1.3012... Val Loss: 1.3397
Epoch: 16/20... Step: 2170... Loss: 1.2957... Val Loss: 1.3409
Epoch: 16/20... Step: 2180... Loss: 1.2864... Val Loss: 1.3392
Epoch: 16/20... Step: 2190... Loss: 1.3118... Val Loss: 1.3375
Epoch: 16/20... Step: 2200... Loss: 1.2867... Val Loss: 1.3379
Epoch: 16/20... Step: 2210... Loss: 1.2488... Val Loss: 1.3356
Epoch: 16/20... Step: 2220... Loss: 1.3045... Val Loss: 1.3347
Epoch: 17/20... Step: 2230... Loss: 1.2820... Val Loss: 1.3364
Epoch: 17/20... Step: 2240... Loss: 1.2836... Val Loss: 1.3372
Epoch: 17/20... Step: 2250... Loss: 1.2630... Val Loss: 1.3352
Epoch: 17/20... Step: 2260... Loss: 1.2825... Val Loss: 1.3335
Epoch: 17/20... Step: 2270... Loss: 1.2861... Val Loss: 1.3336
Epoch: 17/20... Step: 2280... Loss: 1.2977... Val Loss: 1.3323
Epoch: 17/20... Step: 2290... Loss: 1.2926... Val Loss: 1.3296
Epoch: 17/20... Step: 2300... Loss: 1.2601... Val Loss: 1.3316
Epoch: 17/20... Step: 2310... Loss: 1.2743... Val Loss: 1.3291
Epoch: 17/20... Step: 2320... Loss: 1.2729... Val Loss: 1.3294
Epoch: 17/20... Step: 2330... Loss: 1.2713... Val Loss: 1.3246
Epoch: 17/20... Step: 2340... Loss: 1.2874... Val Loss: 1.3312
Epoch: 17/20... Step: 2350... Loss: 1.2879... Val Loss: 1.3265
Epoch: 17/20... Step: 2360... Loss: 1.2965... Val Loss: 1.3260
Epoch: 18/20... Step: 2370... Loss: 1.2629... Val Loss: 1.3246
Epoch: 18/20... Step: 2380... Loss: 1.2692... Val Loss: 1.3262
Epoch: 18/20... Step: 2390... Loss: 1.2696... Val Loss: 1.3236
Epoch: 18/20... Step: 2400... Loss: 1.2902... Val Loss: 1.3238
Epoch: 18/20... Step: 2410... Loss: 1.2847... Val Loss: 1.3220
Epoch: 18/20... Step: 2420... Loss: 1.2635... Val Loss: 1.3206
Epoch: 18/20... Step: 2430... Loss: 1.2722... Val Loss: 1.3211
Epoch: 18/20... Step: 2440... Loss: 1.2606... Val Loss: 1.3211
Epoch: 18/20... Step: 2450... Loss: 1.2511... Val Loss: 1.3213
Epoch: 18/20... Step: 2460... Loss: 1.2694... Val Loss: 1.3203
Epoch: 18/20... Step: 2470... Loss: 1.2676... Val Loss: 1.3217
Epoch: 18/20... Step: 2480... Loss: 1.2463... Val Loss: 1.3199
Epoch: 18/20... Step: 2490... Loss: 1.2457... Val Loss: 1.3207
Epoch: 18/20... Step: 2500... Loss: 1.2605... Val Loss: 1.3168
Epoch: 19/20... Step: 2510... Loss: 1.2662... Val Loss: 1.3150
Epoch: 19/20... Step: 2520... Loss: 1.2656... Val Loss: 1.3171
Epoch: 19/20... Step: 2530... Loss: 1.2692... Val Loss: 1.3148
Epoch: 19/20... Step: 2540... Loss: 1.2761... Val Loss: 1.3143
Epoch: 19/20... Step: 2550... Loss: 1.2437... Val Loss: 1.3183
Epoch: 19/20... Step: 2560... Loss: 1.2609... Val Loss: 1.3140
Epoch: 19/20... Step: 2570... Loss: 1.2408... Val Loss: 1.3145
Epoch: 19/20... Step: 2580... Loss: 1.2809... Val Loss: 1.3152
Epoch: 19/20... Step: 2590... Loss: 1.2485... Val Loss: 1.3115
Epoch: 19/20... Step: 2600... Loss: 1.2433... Val Loss: 1.3135
Epoch: 19/20... Step: 2610... Loss: 1.2484... Val Loss: 1.3141
Epoch: 19/20... Step: 2620... Loss: 1.2462... Val Loss: 1.3125
Epoch: 19/20... Step: 2630... Loss: 1.2364... Val Loss: 1.3132
Epoch: 19/20... Step: 2640... Loss: 1.2556... Val Loss: 1.3065
Epoch: 20/20... Step: 2650... Loss: 1.2525... Val Loss: 1.3078
Epoch: 20/20... Step: 2660... Loss: 1.2579... Val Loss: 1.3088
Epoch: 20/20... Step: 2670... Loss: 1.2617... Val Loss: 1.3068
Epoch: 20/20... Step: 2680... Loss: 1.2638... Val Loss: 1.3071
Epoch: 20/20... Step: 2690... Loss: 1.2567... Val Loss: 1.3108
Epoch: 20/20... Step: 2700... Loss: 1.2569... Val Loss: 1.3071
Epoch: 20/20... Step: 2710... Loss: 1.2297... Val Loss: 1.3076
Epoch: 20/20... Step: 2720... Loss: 1.2294... Val Loss: 1.3075
Epoch: 20/20... Step: 2730... Loss: 1.2266... Val Loss: 1.3059
Epoch: 20/20... Step: 2740... Loss: 1.2127... Val Loss: 1.3054
Epoch: 20/20... Step: 2750... Loss: 1.2264... Val Loss: 1.3083
Epoch: 20/20... Step: 2760... Loss: 1.2137... Val Loss: 1.3063
Epoch: 20/20... Step: 2770... Loss: 1.2554... Val Loss: 1.3057
Epoch: 20/20... Step: 2780... Loss: 1.2731... Val Loss: 1.3009
```

Getting the best model

To set your hyperparameters to get the best performance, you'll want to watch the training and validation losses. If your training loss is much lower than the validation loss, you're overfitting. Increase regularization (more dropout) or use a smaller network. If the training and validation losses are close, you're underfitting so you can increase the size of the network.

Hyperparameters

Here are the hyperparameters for the network.

In defining the model:

- `n_hidden` - The number of units in the hidden layers.
- `n_layers` - Number of hidden LSTM layers to use.

We assume that dropout probability and learning rate will be kept at the default, in this example.

And in training:

- `batch_size` - Number of sequences running through the network in one pass.
- `seq_length` - Number of characters in the sequence the network is trained on. Larger is better typically, the network will learn more long range dependencies. But it takes longer to train. 100 is typically a good number here.
- `lr` - Learning rate for training

Here's some good advice from Andrej Karpathy on training the network. I'm going to copy it in here for your benefit, but also link to where it originally came from.

> ## Tips and Tricks
>
> ### Monitoring Validation Loss vs. Training Loss
>
> If you're somewhat new to Machine Learning or Neural Networks it can take a bit of expertise to get good models. The most important quantity to keep track of is the difference between your training loss (printed during training) and the validation loss (printed once in a while when the RNN is run on the validation data (by default every 1000 iterations)). In particular:
>
> - If your training loss is much lower than validation loss then this means the network might be **overfitting**. Solutions to this are to decrease your network size, or to increase dropout. For example you could try dropout of 0.5 and so on.
> - If your training/validation loss are about equal then your model is **underfitting**. Increase the size of your model (either number of layers or the raw number of neurons per layer)
>
> ### Approximate number of parameters
>
> The two most important parameters that control the model are `n_hidden` and `n_layers`. I would advise that you always use `n_layers` of either 2/3. The `n_hidden` can be adjusted based on how much data you have. The two important quantities to keep track of here are:
>
> - The number of parameters in your model. This is printed when you start training.
> - The size of your dataset. 1MB file is approximately 1 million characters.
>
> These two should be about the same order of magnitude. It's a little tricky to tell. Here are some examples:
>
> - I have a 100MB dataset and I'm using the default parameter settings (which currently print 150K parameters). My data size is significantly larger (100 mil >> 0.15 mil), so I expect to heavily underfit. I am thinking I can comfortably afford to make `n_hidden` larger.
> - I have a 10MB dataset and running a 10 million parameter model. I'm slightly nervous and I'm carefully monitoring my validation loss. If it's larger than my training loss then I may want to try to increase dropout a bit and see if that helps the validation loss.
>
> ### Best models strategy
>
> The winning strategy to obtaining very good models (if you have the compute time) is to always err on making the network larger (as large as you're willing to wait for it to compute) and then try different dropout values (between 0,1). Whatever model has the best validation performance (the loss, written in the checkpoint filename, low is good) is the one you should use in the end.
>
> It is very common in deep learning to run many different models with many different hyperparameter settings, and in the end take whatever checkpoint gave the best validation performance.
>
> By the way, the size of your training and validation splits are also parameters. Make sure you have a decent amount of data in your validation set or otherwise the validation performance will be noisy and not very informative.

Checkpoint

After training, we'll save the model so we can load it again later if we need too. Here I'm saving the parameters needed to create the same architecture, the hidden layer hyperparameters and the text characters.

```
In [17]: # change the name, for saving multiple files
         model_name = 'rnn_x_epoch.net'

         checkpoint = {'n_hidden': net.n_hidden,
                       'n_layers': net.n_layers,
                       'state_dict': net.state_dict(),
                       'tokens': net.chars}

         with open(model_name, 'wb') as f:
             torch.save(checkpoint, f)
```

Making Predictions

Now that the model is trained, we'll want to sample from it and make predictions about next characters! To sample, we pass in a character and have the network predict the next character. Then we take that character, pass it back in, and get another predicted character. Just keep doing this and you'll generate a bunch of text!

A note on the `predict` function

The output of our RNN is from a fully-connected layer and it outputs a **distribution of next-character scores**.

> To actually get the next character, we apply a softmax function, which gives us a *probability* distribution that we can then sample to predict the next character.

Top K sampling

Our predictions come from a categorical probability distribution over all the possible characters. We can make the sample text and make it more reasonable to handle (with less variables) by only considering some K most probable characters. This will prevent the network from giving us completely absurd characters while allowing it to introduce some noise and randomness into the sampled text. Read more about topk, here.

```
In [18]: def predict(net, char, h=None, top_k=None):
                 ''' Given a character, predict the next character.
                     Returns the predicted character and the hidden state.
                 '''

                 # tensor inputs
                 x = np.array([[net.char2int[char]]])
                 x = one_hot_encode(x, len(net.chars))
                 inputs = torch.from_numpy(x)

                 if(train_on_gpu):
                     inputs = inputs.cuda()

                 # detach hidden state from history
                 h = tuple([each.data for each in h])
                 # get the output of the model
                 out, h = net(inputs, h)

                 # get the character probabilities
                 p = F.softmax(out, dim=1).data
                 if(train_on_gpu):
                     p = p.cpu() # move to cpu

                 # get top characters
                 if top_k is None:
                     top_ch = np.arange(len(net.chars))
                 else:
                     p, top_ch = p.topk(top_k)
                     top_ch = top_ch.numpy().squeeze()

                 # select the likely next character with some element of randomness
                 p = p.numpy().squeeze()
                 char = np.random.choice(top_ch, p=p/p.sum())

                 # return the encoded value of the predicted char and the hidden state
                 return net.int2char[char], h
```

Priming and generating text

Typically you'll want to prime the network so you can build up a hidden state. Otherwise the network will start out generating characters at random. In general the first bunch of characters will be a little rough since it hasn't built up a long history of characters to predict from.

```
In [19]: def sample(net, size, prime='The', top_k=None):

             if(train_on_gpu):
                 net.cuda()
             else:
                 net.cpu()

             net.eval() # eval mode

             # First off, run through the prime characters
             chars = [ch for ch in prime]
             h = net.init_hidden(1)
             for ch in prime:
                 char, h = predict(net, ch, h, top_k=top_k)

             chars.append(char)

             # Now pass in the previous character and get a new one
             for ii in range(size):
                 char, h = predict(net, chars[-1], h, top_k=top_k)
                 chars.append(char)

             return ''.join(chars)
```

```
In [20]: print(sample(net, 1000, prime='Anna', top_k=5))

Anna's stairtant."

He had no one thought it to him, had so good himself. She saw a strange of happiness and which had
not true to stury this way," said Varenka, standing and looking into his shoulder.

He felt that, he saw that the mistress was before in the conversation, who taking his face
when they said that, and the staircase said, and he had
not said that the
conversation of the door of the creature with such a
lady was not the most place, to be all he had never spoke of a proposed staying the strength of
hardly to
all his she said in a conversation in the same convincing teather and his brilliant.

"Yes. I have never come to see you and sens that it was as so talking,"
she said, looking at the party, and standing the past of him as he went up
to him,
but that there was no sport of an official tried but in his significance at it, with water to bad suppoming.

"A mistake, to be done, and I am gray in spite of to the positical care, but how talk of you here we taken and feeling. But I c
```

Loading a checkpoint

```
In [21]: # Here we have loaded in a model that trained over 20 epochs `rnn_20_epoch.net`
         with open('rnn_x_epoch.net', 'rb') as f:
             checkpoint = torch.load(f)

         loaded = CharRNN(checkpoint['tokens'], n_hidden=checkpoint['n_hidden'], n_layers=checkpoint['n_layers'])
         loaded.load_state_dict(checkpoint['state_dict'])

Out[21]: <All keys matched successfully>
```

```
In [22]: # Sample using a loaded model
         print(sample(loaded, 2000, top_k=5, prime="And Levin said"))

And Levin said that he were always deciding her to
the man and work of the
sorts, and she was strong at the second thing with something from the same first
one of the
servom. She had bures in her subsuriatable.

Anna down, the frame with his face were conditions, that the stooping came in at the same day of the
courtey standing, she had to see the moment, and the sick minutes of the day of her eyes from a concers of her moveness, the stood was all his high schelising feelings.

"There was in them, it's as he has not been an entrance, and where you are sorry it over, I've been merely for the principal
partician impression to see in him, and there they had anyther word, than I should be any of to tell you and all that I am afraid to than you. What am I all secret," she adked, smiling tryin
g. "I'll did you the pain, and I am great it well, but I could not see him, I'm not so much of
more and said, and I shant a completely son still more."

He stroked up his head, his frightened smiling.
"And I've said such a much and doesn't believe that it were to be said of making of all always by the first sigh.
I can't see you that it something wend away, and she can that shake to see her. I am
not thinking of your sign faresiance; but
then here was the sativating-times ane of their money.
In that was nothing
troubled and
waiting about
that; and we could not see her the cried and to sudden it," he said.

"You're to say to him then? You can't be seen to that single of them. I am all to might through it."

"Oh, then you know how. And there wish to go of a long than all the same in its torthing."

"I am talking to him..."" said Levin.

"What did I see you to say to her? You've tell them to speak of the pool, as you are strayge to her. I have stond to the sense, and how a little woman's stroken to stear. And what are you sor
ry? What as it is so as to mear here in hostility, the softens asked
their persons."

"You must
stoll it is!" said Levin, stood at it, and the matter
to see her and she
holded the
```

```
In [ ]:
```